

Hardware Fundamentals Explained Simply

RTL, Mental Models, and a Full Datapath Walkthrough — in plain words

1. What is RTL?

RTL = **Register Transfer Level**.

In plain words: RTL is a way of *describing hardware* by saying **what data sits in which storage (registers), and how that data moves and gets transformed between clock ticks**. It's written in languages like Verilog/SystemVerilog, and tools turn that description into real silicon — flip-flops, logic gates, and copper wires.

The factory analogy

Imagine a manufacturing workshop:

- **Workers at benches = combinational logic.** They cut, weld, drill. They hold nothing overnight — stop feeding them and they produce nothing.
- **Storage bins = registers.** Shelves that safely hold parts.
- **The shift bell = the clock.** When the bell rings, workers grab parts from Bin A, work on them, and drop the finished item into Bin B.

How software becomes hardware

```
Software (Python / C / Java)
  → "tell the factory what to build"
Assembly (RISC-V)
  → add x5, x6, x7  (a standardized work ticket)
RTL (SystemVerilog)
  → the physical blueprint: which benches, which bins, which conveyor belts
```

- **Software** = a list of operations for a machine to perform.
 - **RTL** = a structural description of a circuit, which gets synthesized into actual gates and flip-flops on a chip.
-

2. What does “Register Transfer” actually mean?

It's the basic loop all digital hardware repeats forever:

1. Register — a row of D flip-flops holding a binary value:

```
Register A = 10 (0x0000000A)
Register B = 20 (0x00000014)
```

2. Transfer — data moves across wires through logic into another register:

```
A (10) ─┐
          │ ──> [ ALU adder ] ──> (30) ──> Register C
B (20) ─┘
```

The full cycle:

```
state storage (source registers)
→ travel along wires
→ combinational logic does the work (ALU, shifters, gates)
→ result routed to the destination
→ destination register latches it on the rising clock edge
```

That's it. That's the “register transfer level” — data living in registers, moving between them on each clock tick.

3. Combinational vs Sequential Logic

Every processor is built from these two kinds of circuits.

Combinational logic — the light switch

- Output depends **only on the present inputs**, after a tiny electrical delay.
- No memory. Nothing is stored.
- **Real-life example:** a wall switch. Flip it → light on instantly. Off → dark instantly.
- **In a CPU:** adders, decoders, multiplexers, sign-extension blocks.

```
A (10) ─┐
          │└─> [ adder ] ─> 30
B (20) ─┘      change A to 15 → output becomes 35 automatically
```

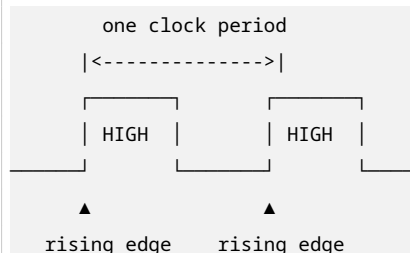
Sequential logic — the camera shutter

- **Stores** data, and only changes state on a clock edge.
- **Real-life example:** a camera. The scene changes constantly, but the shutter only captures a frame at the instant it fires. That frozen frame stays stable until the next shot.
- **In a CPU:** the Program Counter, the register file, pipeline latches.

```
data in (30) ─> [ flip-flop ] ─> stable output Q
                ▲
                └─ only updates on the rising clock edge
```

One-line summary: combinational logic *computes*; sequential logic *remembers*.

4. The Clock — the metronome of the chip



Metronome analogy: musicians stay in time with the conductor's beat.

- **Between ticks:** electrical signals ripple through gates, values are changing, wires are noisy and “in flight.”
- **On the rising edge:** everything settles to a clean 0 or 1, and every register latches its new value at the same instant.

The clock period must be long enough for the *slowest* signal to finish traveling. That single rule decides how fast the whole CPU can run.

5. Single-Cycle vs 3-Stage Pipeline

Single-cycle (Mini-Takshaka): one baker doing everything

The whole instruction — fetch, decode, read registers, ALU, memory, writeback — completes in **one clock period**:

```
one long clock period:
[fetch]→[decode]→[reg read]→[ALU]→[data RAM]→[writeback]
```

Real-life analogy: a single baker who takes an order, mixes, kneads, bakes, packages, and serves — before even looking at the next customer.

- **Good:** zero hazards. Customer 2 never starts until customer 1 is fully done, so nothing can collide.
- **Bad:** everyone waits for the slowest order. A customer asking for water (*addi*) waits the full time of a wedding cake (*lw*). And the clock must stretch to fit the *slowest* path:

```
clock period ≥ Tpc + Tmem + Tdecode + Tregread + Talu + Tdmem + Tmux + Tsetup
```

That's why a single-cycle core is stuck near **~50 MHz** — one tick must cover everything.

3-stage pipeline (Takshaka): the conveyor line

Split the work into 3 stages, separated by registers that hold each instruction's data as it moves:

```
Stage F (fetch) →[R]→ Stage X (execute) →[R]→ Stage W (memory/writeback)
```

Three bakers in a row: one takes orders, one preps, one bakes and packages.

```
cycle 1: instr 1 fetch
cycle 2: instr 2 fetch | instr 1 execute
cycle 3: instr 3 fetch | instr 2 execute | instr 1 writeback
```

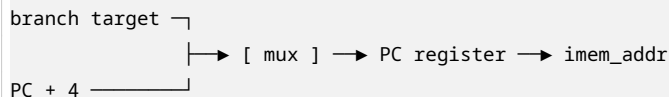
- **Gain:** each stage's delay is about 1/3 of the single-cycle path, so the clock runs **3–4× faster** (~200 MHz).
 - **Cost:** now 3 instructions are “in flight” at once, so we need hazard handling:
 - **W→X forwarding:** pass results straight back so the next instruction doesn't wait.
 - **Branch prediction:** guess branches so the fetch stage doesn't idle.
-

6. The 5 Core Blocks of the Datapath

1. Program Counter (address engine)	2. Instruction Decoder (control)	3. Register File	4. ALU / AGU (math) (address gen)	5. Writeback Mux (commit)
--	--	---------------------	---	---------------------------------

Block 1 — Program Counter (PC)

- A 32-bit register holding the address of the current instruction.
- Normally steps forward: $\text{next PC} = \text{PC} + 4$.
- On a taken branch, a mux feeds the calculated target instead:



Block 2 — Instruction Decoder

- Pure combinational logic that slices the 32-bit instruction into fields and raises control lines.

```
instruction bits → funct7 | rs2 | rs1 | funct3 | rd | opcode
                  → picks the ALU operation (ADD/SUB/AND/OR...)
                  → asserts reg_write (allow writing a register)
                  → asserts dmem_we (allow writing memory)
```

Block 3 — Register File (x0–x31)

- 32 registers, 32 bits each.
- **Two read ports:** give it rs1/rs2 numbers, get the data out *combinationally* (no clock needed).
- **One write port:** on the rising clock edge, if `reg_write` is on, `wb_data` lands in register `rd`.
- **x0 is special:** hardwired to 0. Reads give 0, writes are ignored.

Block 4 — ALU / AGU

- The math engine. Input A = register data 1. Input B = a mux choosing between register data 2 (R-type) or the immediate (I-type).
- **AGU (Address Generation Unit):** for loads/stores it computes the memory address: **effective address** = `rs1 + immediate`

Block 5 — Writeback Mux

- A selector that picks *what* gets written back to the register file:

```
wb_data = (is_load) ? dmem_rdata : alu_result
```

Load → memory data; anything else → the ALU result.

7. Full Walkthrough: `add x5, x6, x7` in hardware

Instruction: `add x5, x6, x7` (meaning: `x5 = x6 + x7`) Initial state: `x6 = 10`, `x7 = 20`, `PC = 0x00001000`

Step 1 — Fetch

- PC outputs address `0x00001000` onto the instruction-memory bus.
- Memory returns the 32-bit machine code: `0x007302B3`.
- In parallel, an adder computes $PC + 4 = 0x00001004$.

Step 2 — Decode

Break the bits of `0x007302B3` into fields:

```
funct7=0000000 | rs2=00111 (x7) | rs1=00110 (x6) | funct3=000 | rd=00101 (x5) | opcode=0110011 (R-type)
```

The decoder now knows: read x6 and x7, add them, write the result to x5. It sets `ALU = ADD`, `reg_write = 1`, `dmem_we = 0`.

Step 3 — Register read

- `rs1 = 6` → register file outputs `10` on read port 1.
- `rs2 = 7` → register file outputs `20` on read port 2.

Step 4 — Execute

- The ALU input mux picks register data (not an immediate) for input B.
- The adder computes $10 + 20 = 30$ (`0x0000001E`).
- Since it's not a branch, `branch_taken` stays 0, so PC will get $PC + 4$.

Step 5 — Writeback

- The writeback mux sees this is not a load, so it selects the ALU result: `30`.
- **Rising clock edge:** `30` is latched into x5, and PC latches `0x00001004`.
- Instruction done. Next cycle begins with the next instruction.

That's the entire life of one instruction — five hops, one clock tick.

8. Architectural Evolution: the summary table

	Mini-Takshaka (single-cycle)	Ibex (2-stage)	Takshaka (3-stage)
Pipeline depth	1 (no latches between)	2	3 (F→X→W)
CPI	1.0	~1.2	~1.0
Clock speed	Low (~50 MHz)	Moderate (~120 MHz)	High (~200+ MHz)
Hazards	None (sequential execution)	Interlocked stalling	Full W→X forwarding, no load-use stalls
Branch penalty	0 (PC decided same cycle)	1 cycle	1 cycle flush (hidden by gshare predictor)
CoreMark/MHz	Low (clock-limited)	~1.43	2.68

Conclusion

Watching how signals, muxes, and flip-flops cooperate during one clock cycle makes the big ideas click: deeper pipelining buys speed, forwarding kills data hazards, and branch prediction pays for the pipeline's one weakness. Those three ideas are exactly what separates Mini-Takshaka from Takshaka.

Study notes by Shubhi Rai. Companion to the Mini-Takshaka core notes and the Takshaka core study.